

Desenvolvimento Seguro de Aplicações Web

TP1



Instituto Infnet

Aluno: Breno Lobato Vila Nova

Curso: Engenharia de software

Cidade: Rio de Janeiro

Escola: ESTI

Data: 16/08/2026

Professor: Fabiano de Moraes Domingues

Questões

Exercício 1

Tarefa 1 - Crie um ambiente virtual Python isolado para o projeto eventos-api usando virtualenv e instale o FastAPI e o uvicorn via pip, registrando as dependências em um arquivo requirements.txt.

Resposta:

```
[→ tp1 git:(main) uv venv .venv
Using CPython 3.12.13
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
[→ tp1 git:(main) source .venv/bin/activate
(.venv) → tp1 git:(main) uv pip install "fastapi[standard]"
Resolved 43 packages in 27ms
Installed 43 packages in 23ms
+ annotated-doc==0.0.5
+ annotated-types==0.8.0
+ anyio==4.14.2
+ certifi==2026.7.22
+ click==8.4.2
+ detect-installer==0.1.0
+ dnspython==2.8.0
+ email-validator==2.3.0
+ fastapi==0.141.1
+ fastapi-cli==0.0.32
```

```
+ watchfiles==1.2.0
+ websockets==17.0.1
(.venv) → tp1 git:(main) uv pip install "uvicorn[standard]"
Checked 1 package in 18ms
(.venv) → tp1 git:(main) ×
```

Tarefa 2 - Estruture uma aplicação FastAPI mínima em main.py, com uma rota GET / que retorna uma mensagem de status do serviço.

Resposta:

Resposta no código

Tarefa 3 - Suba o servidor com uvicorn em modo de hot-reload e capture, em texto ou print, a resposta da rota acessada via navegador ou httpie.

Resposta:

```
(.venv) → app git:(main) × fastapi dev

⚡ Starting FastAPI in development mode

🔗 Using import string: main:app (auto-discovered, use --verbose to learn more)

💡 You can configure an entrypoint in pyproject.toml for this app with:

[tool.fastapi]
entrypoint = "main:app"

🌐 Server started at http://127.0.0.1:8000
Documentation at http://127.0.0.1:8000/docs

Logs:

| Will watch for changes in these directories:
| ['/Users/kick/workspace/university/ia_agentes/tps/security_development/tp1/app']
| Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
| Started reloader process [18155] using WatchFiles
| Started server process [18164]
| Waiting for application startup.
| Application startup complete.
| 127.0.0.1:63593 - "GET / HTTP/1.1" 200
| 127.0.0.1:63596 - "GET / HTTP/1.1" 200
| 127.0.0.1:63598 - "GET / HTTP/1.1" 200
```

```
(.venv) → app git:(main) × fastapi dev

⚡ Starting FastAPI in development mode

🐍 Using import string: main:app (auto-discovered, use --verbose to learn more)

💡 You can configure an entrypoint in pyproject.toml for this app with:

[tool.fastapi]
entrypoint = "main:app"

🌐 Server started at http://127.0.0.1:8000
Documentation at http://127.0.0.1:8000/docs

Logs:

| Will watch for changes in these directories:
| [' /Users/kick/workspace/university/ia_agentes/tps/security_development/tp1/app']
| Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
| Started reloader process [18155] using WatchFiles
| Started server process [18164]
| Waiting for application startup.
| Application startup complete.
| 127.0.0.1:63593 - "GET / HTTP/1.1" 200
| 127.0.0.1:63596 - "GET / HTTP/1.1" 200
| 127.0.0.1:63598 - "GET / HTTP/1.1" 200
```

Exercício 2

Tarefa 1 - Implemente um APIRouter dedicado ao recurso eventos, com pelo menos três operações RESTful (por exemplo, listar eventos, criar evento, obter evento por id).

Resposta:

Resposta no código

Tarefa 2 - Registre esse router na aplicação principal usando include_router, mantendo o arquivo main.py livre de definições de rota específicas do domínio de eventos.

Resposta:

Resposta no código

Tarefa 3 - Justifique, em um parágrafo curto, por que separar routers por recurso facilita a manutenção de uma API que crescerá para múltiplos domínios.

Resposta:

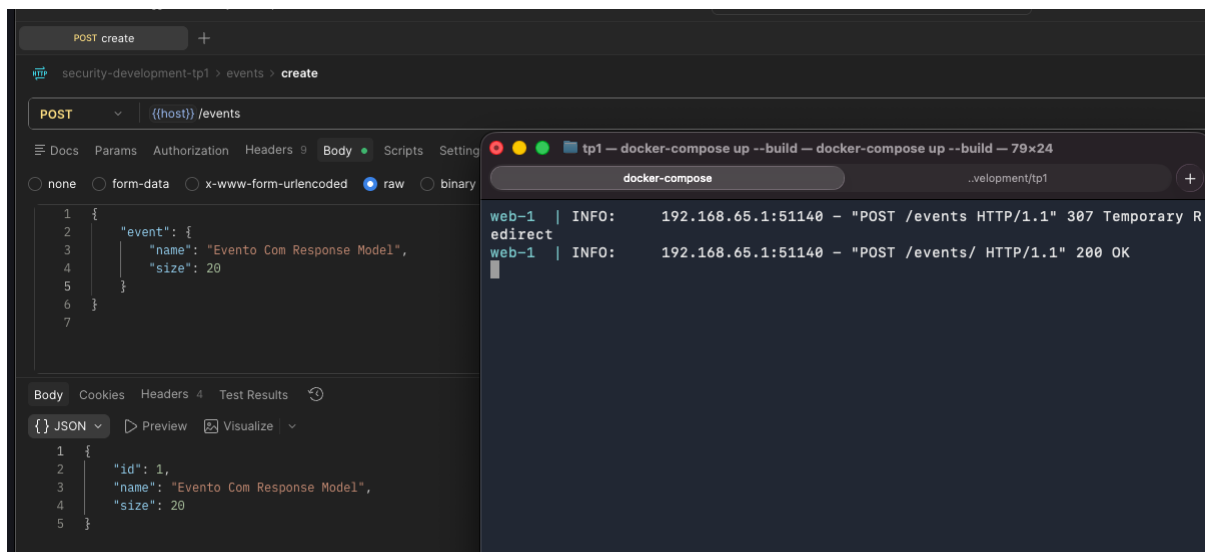
Separar os routes por recurso, por exemplo events(serve /events) e users(serve /users), é importante para manter a facilidade e rapidez para achar os componentes router responsáveis por servir aquele recurso, principalmente quando se sabe que no

futuro existirão outros recursos, e mante-los no mesmo lugar além de dificultar a manutenção também aumenta o risco de side-effect.

Exercício 3

Tarefa 1 - Construa um response_model Pydantic para o endpoint de criação de evento, incluindo apenas os campos que devem ser visíveis ao cliente da API (excluindo campos internos como token de auditoria).

Resposta:

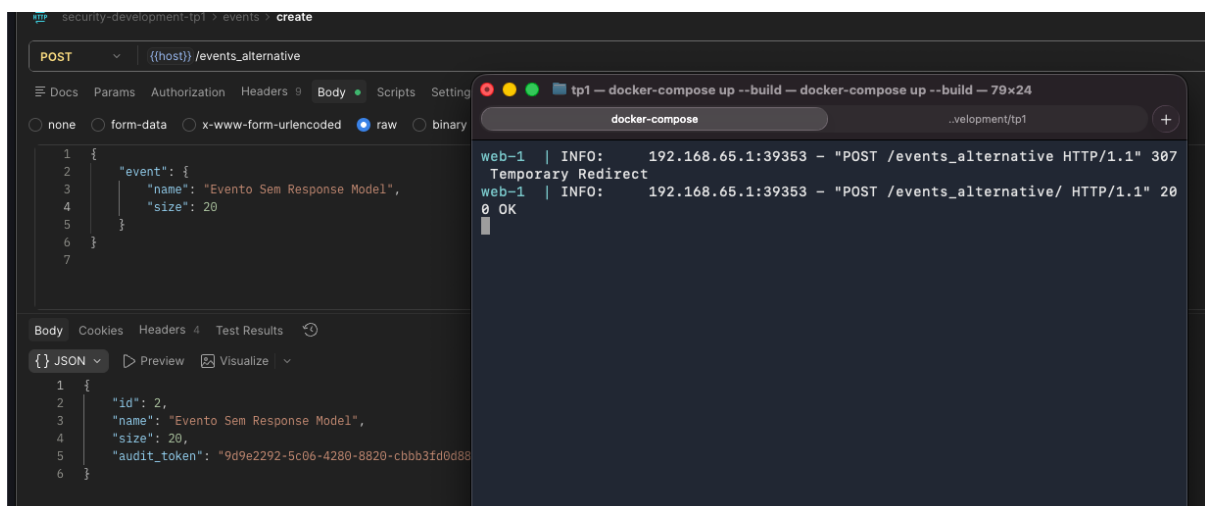


The screenshot shows a REST client interface with a POST request to `POST {{host}}/events`. The request body is a JSON object: `{ "event": { "name": "Evento Com Response Model", "size": 20 } }`. The response is a 200 OK status with a JSON body: `{ "id": 1, "name": "Evento Com Response Model", "size": 20 }`. A terminal window in the background shows the command `docker-compose up --build` and the output of the request.

Tarefa 2 - Escreva uma versão alternativa do mesmo endpoint sem response_model definido e compare, em texto, a resposta JSON retornada nos dois casos.

Resposta:

Como podemos ver nos prints de responses, o response model filtra o dicionário retornado para incluir somente os valores presentes no response model.



The screenshot shows a REST client interface with a POST request to `POST {{host}}/events_alternative`. The request body is a JSON object: `{ "event": { "name": "Evento Sem Response Model", "size": 20 } }`. The response is a 200 OK status with a JSON body: `{ "id": 2, "name": "Evento Sem Response Model", "size": 20, "audit_token": "9d9e2292-5c06-4280-8820-cbbb3fd0d88" }`. A terminal window in the background shows the command `docker-compose up --build` and the output of the request.

Tarefa 3 - Explique, em até cinco linhas, qual dado sensível seria exposto na versão sem response_model e qual seria o impacto prático dessa exposição para o organizador do evento.

Resposta:

Dependendo de quais dados aquele endpoint estivesse retornando em um caso real, não filtrar a resposta poderia vazar informações do evento que não estariam disponíveis para o público, como por exemplo uma participação especial que apareceria no evento ou até mesmo vazar os produtos que seriam lançados no evento antes mesmo da data.

Exercício 4

Tarefa 1 - Reorganize o eventos-api em pelo menos três módulos separados: routes, models e database (ou nomes equivalentes), movendo o código existente para os arquivos corretos.

Resposta:

```
[→ app git:(tp1/exercise-4) × ls
databases          models          routes
main.py            requirements.txt
[→ app git:(tp1/exercise-4) × ls databases
events_api_database.py
[→ app git:(tp1/exercise-4) × ls models
events users
[→ app git:(tp1/exercise-4) × ls models/events
new_event_model.py response_model.py
[→ app git:(tp1/exercise-4) × ls routes
base_router.py      events_router.py
events_alternative_router.py
→ app git:(tp1/exercise-4) × █
```

Tarefa 2 - Garanta que a aplicação continue funcional após a reorganização, testando as rotas implementadas nos exercícios anteriores.

Resposta:

The screenshot shows a REST client interface with a GET request to `{{host}}/events`. The response is a JSON object with a `result` array containing one event. The event has an `id` of 1, a `name` of "Evento Sem Response Model", a `size` of 20, and an `audit_token`. The status is 200 OK. In the background, a terminal window shows Docker logs for the `web-1` service, indicating successful GET requests to `/events` and `/events/1`.

```
GET {{host}}/events

Body
JSON
1 {
2   "result": [
3     {
4       "id": 1,
5       "name": "Evento Sem Response Model",
6       "size": 20,
7       "audit_token": "53b4d07e-697a-408a-98e4-99c6e97bb332"
8     }
9   ]
10 }
```

```
web-1 | INFO: 172.18.0.1:64648 - "GET /events HTTP/1.1" 307 Temporary Redi
rect
web-1 | INFO: 172.18.0.1:64648 - "GET /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "POST /events HTTP/1.1" 307 Temporary Red
irect
web-1 | INFO: 172.18.0.1:64648 - "POST /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "POST /events HTTP/1.1" 307 Temporary Red
irect
web-1 | INFO: 172.18.0.1:64648 - "POST /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "GET /events/1 HTTP/1.1" 200 OK
```

The screenshot shows a REST client interface with a POST request to `{{host}}/events`. The request body is a JSON object with an `event` object containing `name` "Evento Com Response Model" and `size` 20. The response is 200 OK. The terminal window shows Docker logs for the `web-1` service, indicating successful POST requests to `/events` and `/events/1`.

```
POST {{host}}/events

Body
JSON
1 {
2   "event": {
3     "name": "Evento Com Response Model",
4     "size": 20
5   }
6 }
```

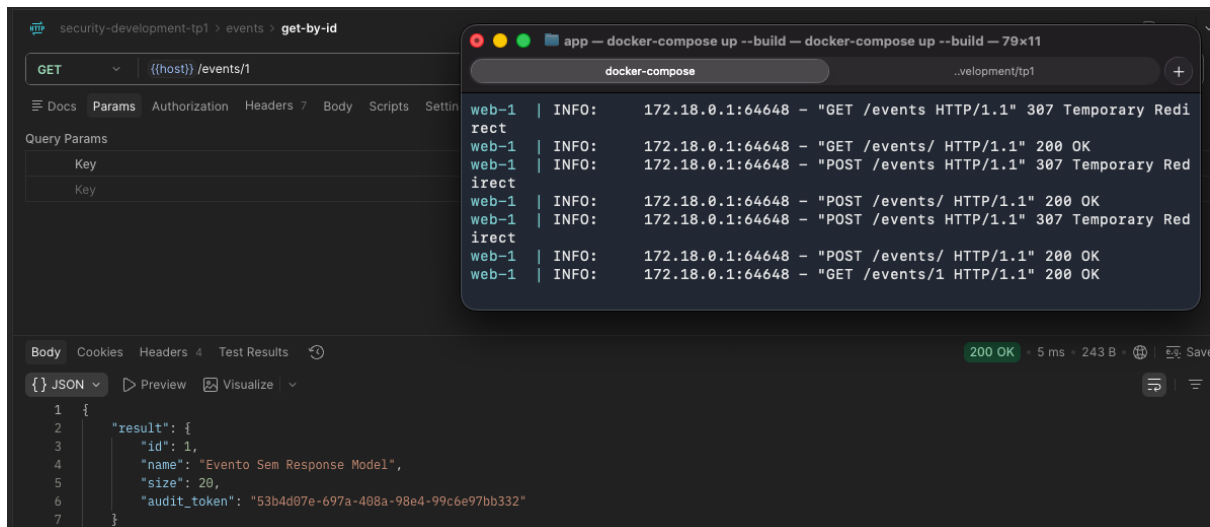
```
web-1 | INFO: 172.18.0.1:64648 - "GET /events HTTP/1.1" 307 Temporary Redi
rect
web-1 | INFO: 172.18.0.1:64648 - "GET /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "POST /events HTTP/1.1" 307 Temporary Red
irect
web-1 | INFO: 172.18.0.1:64648 - "POST /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "POST /events HTTP/1.1" 307 Temporary Red
irect
web-1 | INFO: 172.18.0.1:64648 - "POST /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "GET /events/1 HTTP/1.1" 200 OK
```

The screenshot shows a REST client interface with a POST request to `{{host}}/events_alternative`. The request body is a JSON object with an `event` object containing `name` "Evento Sem Response Model" and `size` 20. The response is 200 OK. The terminal window shows Docker logs for the `web-1` service, indicating successful POST requests to `/events` and `/events/1`.

```
POST {{host}}/events_alternative

Body
JSON
1 {
2   "event": {
3     "name": "Evento Sem Response Model",
4     "size": 20
5   }
6 }
```

```
web-1 | INFO: 172.18.0.1:64648 - "GET /events HTTP/1.1" 307 Temporary Redi
rect
web-1 | INFO: 172.18.0.1:64648 - "GET /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "POST /events HTTP/1.1" 307 Temporary Red
irect
web-1 | INFO: 172.18.0.1:64648 - "POST /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "POST /events HTTP/1.1" 307 Temporary Red
irect
web-1 | INFO: 172.18.0.1:64648 - "POST /events/ HTTP/1.1" 200 OK
web-1 | INFO: 172.18.0.1:64648 - "GET /events/1 HTTP/1.1" 200 OK
```



Tarefa 3 - Documente, em um arquivo README.md curto, a responsabilidade de cada módulo criado.

Resposta:

Resposta no código

Exercício 5

Tarefa 1 - Integre o Jinja2 à aplicação FastAPI, criando uma rota que renderiza um template HTML listando os eventos cadastrados.

Resposta:

Resposta no código

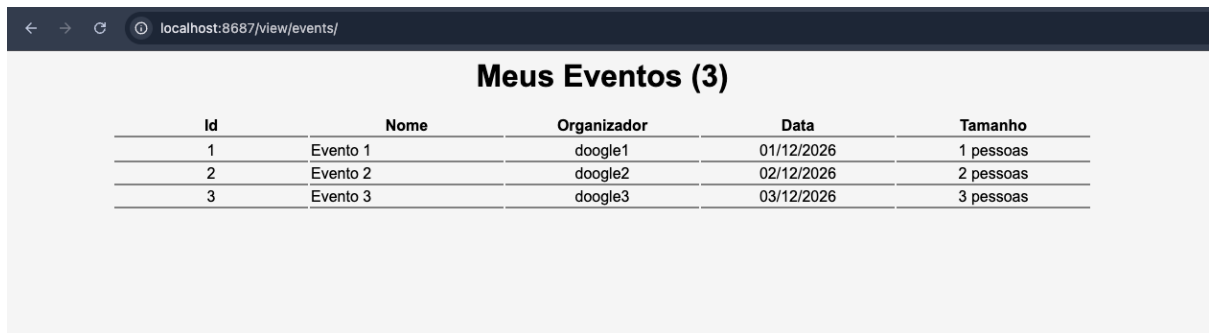
Tarefa 2 - Utilize o motor de templates para exibir dinamicamente pelo menos três campos de cada evento (nome, data, organizador).

Resposta:

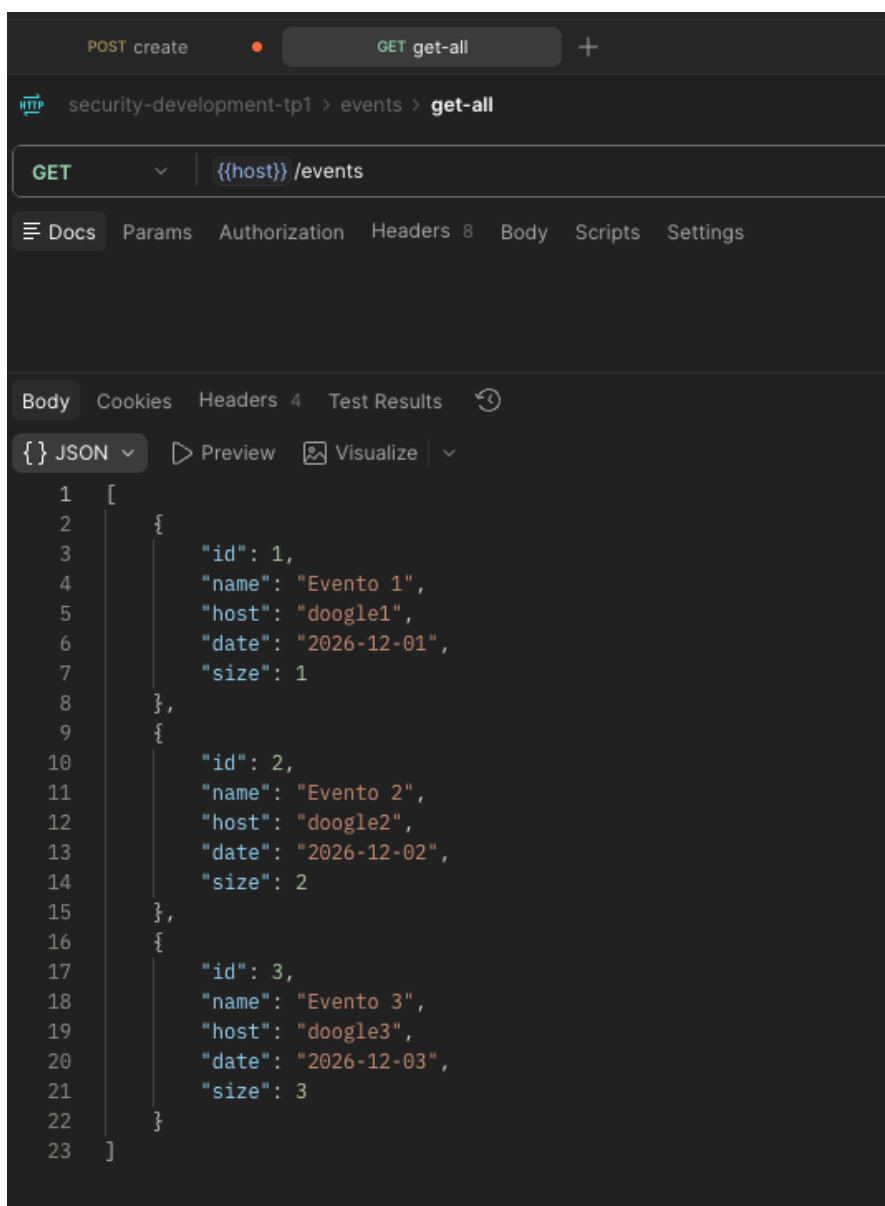
Resposta no código

Tarefa 3 - Confirme que a rota original em JSON (GET /eventos) continua funcionando normalmente, sem interferência da nova rota HTML.

Resposta:



Id	Nome	Organizador	Data	Tamanho
1	Evento 1	doogle1	01/12/2026	1 pessoas
2	Evento 2	doogle2	02/12/2026	2 pessoas
3	Evento 3	doogle3	03/12/2026	3 pessoas



```
POST create
GET get-all

security-development-tp1 > events > get-all

GET {{host}} /events

Body Cookies Headers 4 Test Results
{} JSON Preview Visualize
1 [
2   {
3     "id": 1,
4     "name": "Evento 1",
5     "host": "doogle1",
6     "date": "2026-12-01",
7     "size": 1
8   },
9   {
10    "id": 2,
11    "name": "Evento 2",
12    "host": "doogle2",
13    "date": "2026-12-02",
14    "size": 2
15  },
16  {
17    "id": 3,
18    "name": "Evento 3",
19    "host": "doogle3",
20    "date": "2026-12-03",
21    "size": 3
22  }
23 ]
```

Exercício 6

Tarefa 1 - Reproduza o comportamento relatado pelo QA cadastrando um evento com um payload contendo uma tag ``<script>`` no campo nome e observe como o Jinja2 trata esse conteúdo por padrão.

Resposta:

The screenshot shows a web application titled "Meus Eventos (1)". It contains a table with the following data:

Id	Evento	Organizador
1	Evento <code><script>alert("hello")</script></code>	doogle1

Below the table, the browser's developer tools are open, showing the source code of the page. The code is as follows:

```
<!-- ... -->
</head>
</>
<div class="text-center">Meus Eventos (1)</div>
<div class="flex-center">
  <table>
    <thead>
      <tr>
        <th class="text-center event-data">Id</th>
        <th class="text-start event-data">Evento<script>alert("hello")</script></th>
        <th class="text-center event-data">Organizador</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td class="text-center event-data">1</td>
        <td class="text-start event-data">Evento &lt;script&gt;alert("hello")&lt;/script&gt;</td>
        <td class="text-center event-data">doogle1</td>
        <td class="text-center event-data">10 pessoas</td>
      </tr>
    </tbody>
  </table>
</div>
```

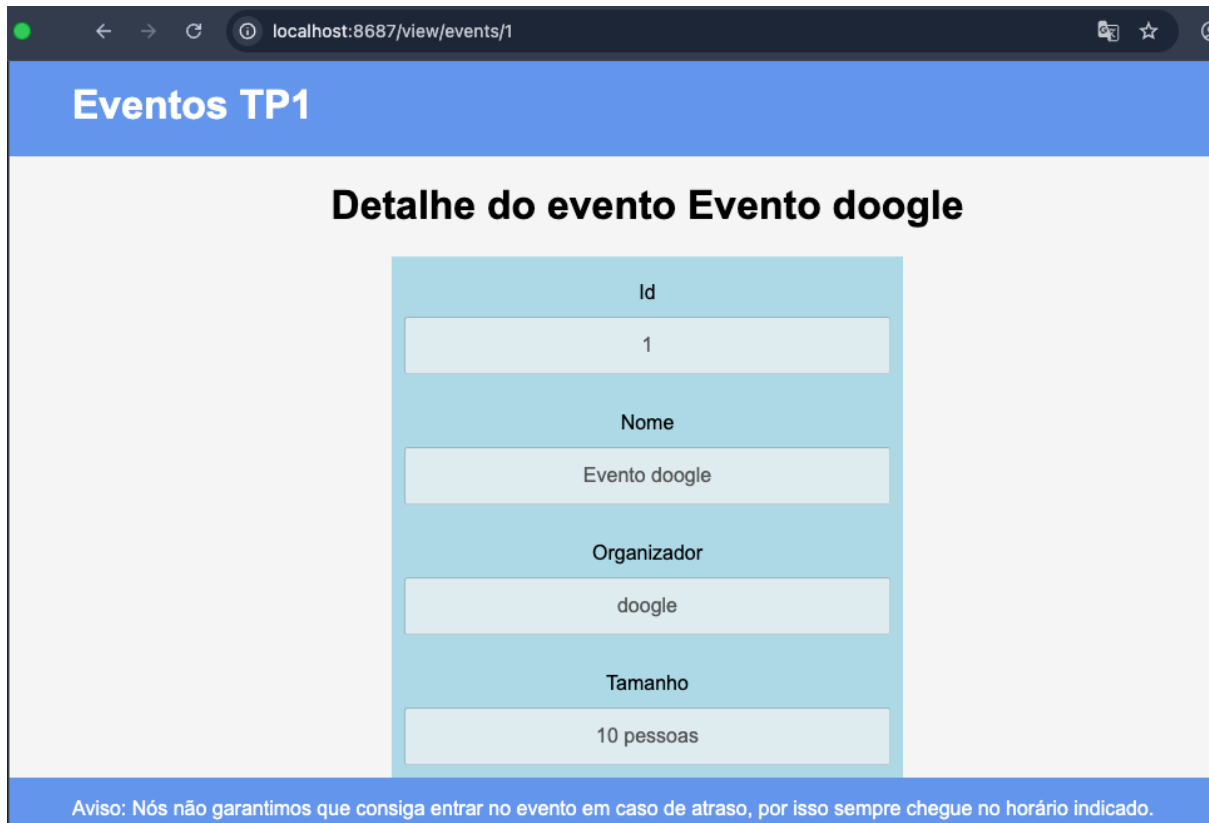
Tarefa 2 - Explique, em até seis linhas, por que a renderização de input do usuário sem escape adequado representa um risco de XSS, e o que o auto-escape do Jinja2 faz para mitigar isso.

Resposta:

O XSS (Cross-Site Scripting) é uma falha de segurança em que se baseia em inserir um código malicioso no input não tratado, e quando esse input é renderizado na tela sem o escape causa a execução daquele script e criando uma vulnerabilidade. O Jinja2, como mostrando no print, troca o character "<" e ">" por respectivamente "<" e ">", isso faz com que no momento da exibição na tela ele apareça corretamente, mas é ignorado como uma tag script válida e por isso não executa o código ali dentro.

Tarefa 3 - Implemente herança de templates com um template base contendo cabeçalho e rodapé, e faça a página de listagem e a página de detalhes de evento herdarem dele.

Resposta:



The screenshot shows a web browser window with the address bar displaying 'localhost:8687/view/events/1'. The page has a blue header with the text 'Eventos TP1'. Below the header, the main content area has a title 'Detalhe do evento Evento doogle'. Underneath the title, there is a light blue box containing four rows of event details, each with a label and a value in a white input field:

Id
1
Nome
Evento doogle
Organizador
doogle
Tamanho
10 pessoas

At the bottom of the page, there is a blue footer with the following text: 'Aviso: Nós não garantimos que consiga entrar no evento em caso de atraso, por isso sempre chegue no horário indicado.'

Eventos TP1					
Meus Eventos (1)					
Id	Nome	Organizador	Data	Tamanho	
1	Evento doogle	doogle	01/12/2026	10 pessoas	Detalhes

Aviso: Nós não garantimos que consiga entrar no evento em caso de atraso, por isso sempre chegue no horário indicado.

Exercício 7

Tarefa 1 - Para a aplicação eventos-api construída nos exercícios anteriores, avalie um exemplo concreto de confidencialidade, um de integridade e um de disponibilidade, apontando o que já está implementado e o que ainda está em aberto.

Resposta:

- **Confidencialidade:** Atualmente nosso sistema não possui recursos de controle de acesso.
- **Integridade:** O nosso sistema não permite atualizar um evento depois de criado, logo não existe risco de que o dado que está tenha sido alterado.
- **Disponibilidade:** Atualmente não existem dependência de serviços externos para o funcionamento da api.

Tarefa 2 - Aponte pelo menos uma lacuna real observável no código atual para cada um dos três pilares da tríade CIA.

Resposta:

- **Confidencialidade:** Falta de login.
- **Integridade:** Falta de um armazenamento não volátil.
- **Disponibilidade:** Não existe recuperação de dados após reiniciar a api.

Tarefa 3 - Registre essa avaliação em uma tabela simples com as colunas Pilar, Situação atual e Lacuna.

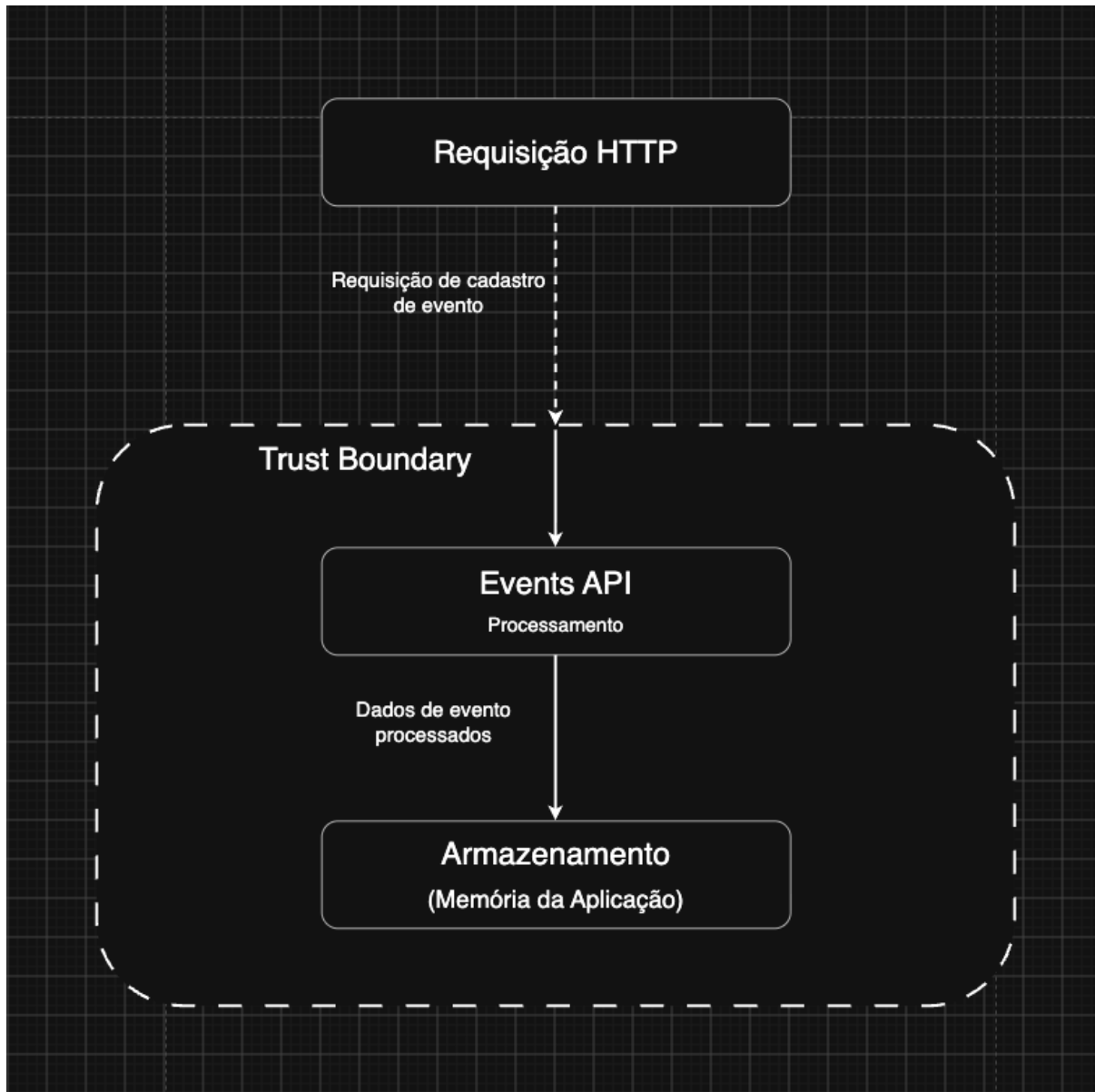
Resposta:

Pilar	Situação atual	Lacuna
Confidencialidade	Não temos recursos nesse pilar	Não existe proteção para acesso
Integridade	Os eventos armazenados são imutáveis	O armazenamento de eventos é volátil
Disponibilidade	Não dependemos de serviços externos	Não existe como recuperar os dados após reiniciar a api

Exercício 8

Tarefa 1 - Construa um DFD básico do eventos-api, representando pelo menos: entrada de dados do usuário, processamento na API e armazenamento (mesmo que simulado), destacando as trust boundaries entre eles.

Resposta:



Tarefa 2 - Associe, em uma tabela, os frameworks OWASP, NIST SSDF e MITRE a um controle de segurança concreto já discutido nos exercícios anteriores (por exemplo, response_model, escape de templates, virtualenv).

Resposta:

Framework	Controle de Segurança
OWASP	response_model
NIST SSDF	virtualenv
MITRE	Escape de Template

Tarefa 3 - Aponte no DFD pelo menos um fluxo de dados sensível (por exemplo, dados do organizador) e a trust boundary que ele atravessa.

Resposta:

No atual escopo da aplicação não existem dados sensíveis vindo de fora, o mais perto de um dado sensível existente é o audit_token.

Mas para exemplificar, se em um futuro tivéssemos mais de 1 usuário inscrito no mesmo evento teríamos que implementar proteções para que não fossem retornados eventos que não pertencem àquele usuário da request.

Fluxo sensível — Cenário futuro

